Submission Worksheet

Submission Data

Course: IT202-007-F2025
Assignment: IT202 - Milestone 3
Student: Roshan T. (rt524)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 12/4/2025 2:16:23 PM
Updated: 12/6/2025 7:09:15 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT202-007-F2025/it202-milestone-3/grading/rt524
View Link: https://learn.ethereallab.app/assignment/v3/IT202-007-F2025/it202-milestone-3/view/rt524

Instructions

1. Refer to Milestone3 of this doc: https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone3 branch
    1. `git checkout Milestone3` (ensure proper starting branch)
    2. `git pull origin Milestone3` (ensure history is up to date)
5. Fill out the below worksheet
    - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    - Ensure proper styling is applied to each page
    - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone3`
    4. On Github merge the pull request from `Milestone3` to `dev`
    5. On Github create a pull request from `dev` to `prod` and immediately merge. (This will trigger the `prod` deploy to make the heroku prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout dev`
11. `git pull origin dev`

Section #1: (3 pts.) Api Data

Progress: 100%

Task #1 (1 pt.) - Concept of Data Association

Progress: 100%

Details:

- What's the concept of your data association to users? (examples: favorites, wish list, purchases, assignment, etc)
- Describe with a few sentences

Your Response:

The concept of association is a favorites/bookmarks page. Users will be able to save certain jobs to their "My Jobs" page so that they can view them when needed. Operates like a favorites because users can choose what to assign to themselves.

Saved: 12/4/2025 2:17:09 PM

Task #2 (1 pt.) - Data Updates

Progress: 100%

Details:

- When an associated entity is updated (manually or API) how is the association affected?
  - Does the user see the old version of the data?
  - Does the user see the new version of the data?
  - Does the user need to have data re-associated or remapped?
- Explain why.

Your Response:

When an associated entity is updated in any way, the user sees the updated data. The job_id is referenced to the main Jsearch table which means any updated job_id in any way will reflect on the saved job from the user. The user does not have to reassociate the data as the new changes will be reflected in their saved jobs.

Saved: 12/4/2025 2:19:45 PM

Task #3 (1 pt.) - Handling Association

Progress: 100%

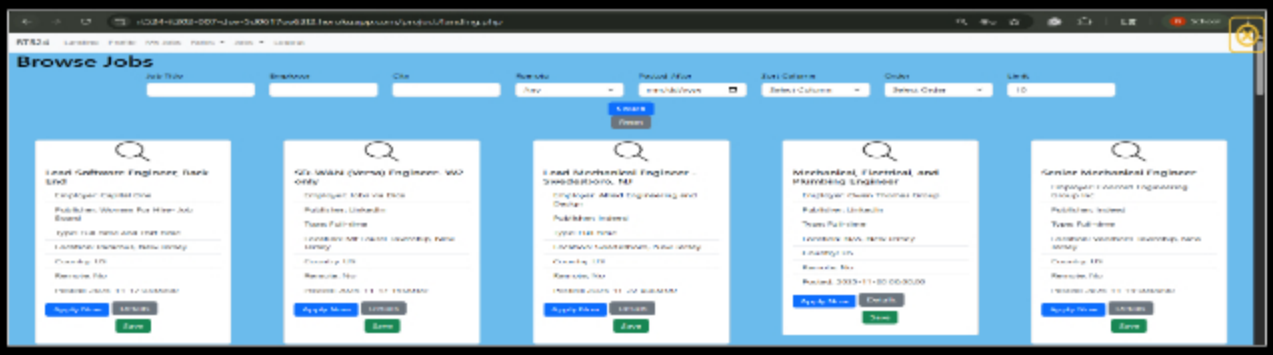
Part 1:

Progress: 100%

Details:

- Show an example page of where a user can get data associated with them
- Ensure heroku dev url is visible

- Caption if this is a user-facing page or admin page



save button on landing page, user facing page

```
<!-- Save jobs button, only interactable if not saved -->
<?php if (is_logged_in()) : ?>
    <?php if ($is_saved): ?>
        <form method="POST" action="<?php echo get_url('unsave_job.php'); ?>">
            <input type="hidden" name="job_id" value="<?php se($data, 'job_id'); ?>">
            <button type "submit" class "btn btn success mt 2">Unsave</button>
        </form>
    <?php else: ?>
        <form method="POST" action="<?php echo get_url('save_job.php'); ?>">
            <input type="hidden" name="job_id" value="<?php se($data, 'job_id'); ?>">
            <button type="submit" class="btn btn-success mt-2">Save</button>
        </form>
    <?php endif; ?>
<?php endif; ?>
    </div>
</div>
<?php endif; ?>
```

job card save code

```
<?php
// rubric 12/2
// check if job saved already
$is_saved = false;
if (is_logged_in()) {
    try {
        $db = getDB();
        $stmt = $db->prepare("
            SELECT 1 FROM [illegible]
            WHERE user_id = :uid
              AND job_id = :jid
              AND is_active = 1
        ");
        $stmt->execute([
            ":uid" => get_user_id(),
            ":jid" => $data["job_id"]
        ]);
        $is_saved = $stmt->fetch() ? true : false;
    } catch (Exception $e) {
        error_log("Error checking saved jobs: " . var_export($e, true));
    }
}
?>
```

job card already saved code

Saved: 12/6/2025 6:58:27 PM

Part 2:

Progress: 100%

Details:

- Describe the process of associating data with the user.
- Can it be toggled, or is it applied once?

Your Response:

The save button directs to save_job.php which runs a query that associates the user ID to the job ID, and if it already exists, unsets the item. This is a toggle that can go from save to unsave, the query handles both properly based on the is_active property.

https://rt524-it202-007-prod-a195f178bfec.herokuapp.com/project/landing.php

Saved: 12/6/2025 6:58:27 PM

Section #2: (6 pts.) Associations

Progress: 100%

Task #1 (1.50 pts.) - Logged-in User's Associated Entities

Progress: 100%

Details:

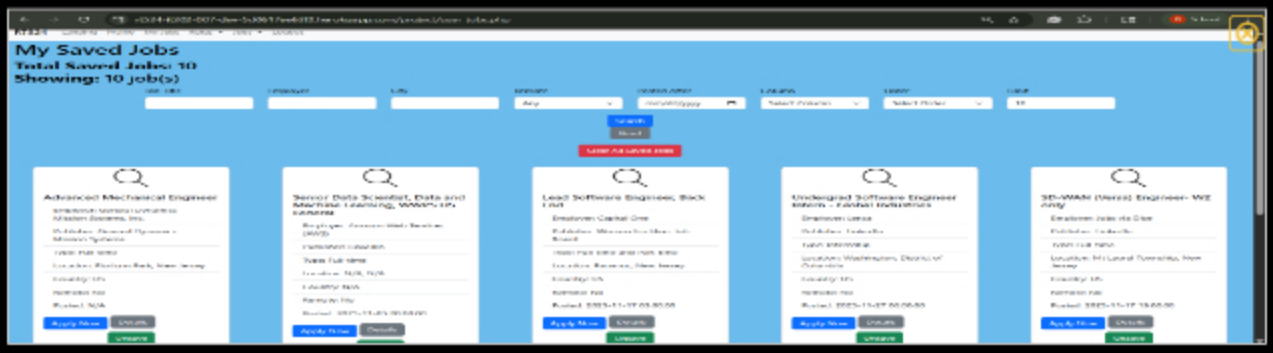
- Each line item should have a logical summary
- Each line item should have a link/button to a single view page of the entity
- Each line item should have a link/button to a delete action of the relationship (doesn't delete the entity or user, just the relationship)
- The page should have a link/button to remove all associations for the particular user
- The page should have a section for stats (number of results and total number possible based on the query filters)
- The page should have logical options for filtering/sorting
  - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)
  - A filter with no matching records should show "no results available" or equivalent

Part 1:

Progress: 100%

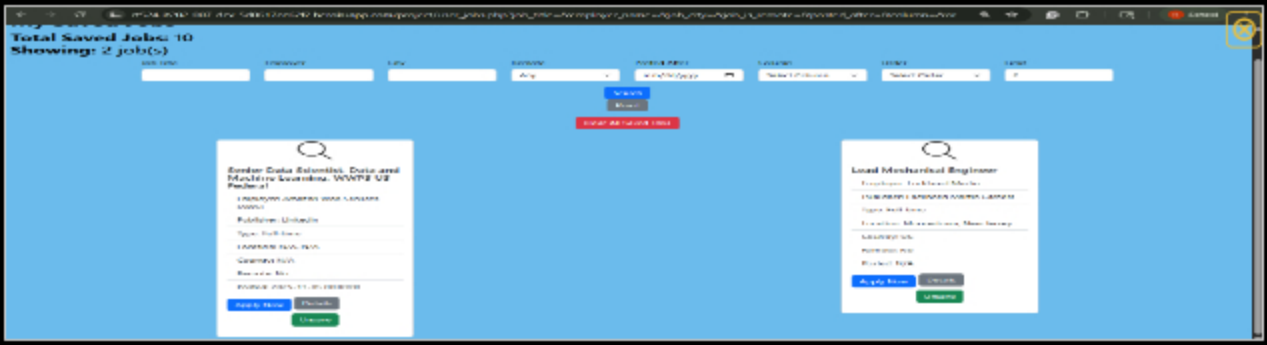
Details:

- Show a few examples of this page from heroku dev with various filters applied
- Ensure heroku dev url is visible
- Ensure each requirement is visible



screenshot of "saved jobs" page

filter applied



limit applied

user saved jobs code

user saved jobs code part 2

user saved jobs code part 3

user saved jobs code part 4

user saved jobs code form

Saved: 12/6/2025 6:59:42 PM

Part 2:

Progress: 100%

Details:

- Describe how you solved showing the particular association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

This association page uses a query that matches the current user ID and gets any jobs where user_id is 1. Then displays them in the form of a job card like landing.php. The stats were solved by how many rows were returned and the filter logic is the same as landing, where the filters are a form and then applied to a query.

https://rt524-it202-007-prod-a195f178bfec.herokuapp.com/project/user_jobs.php

Saved: 12/6/2025 6:59:42 PM

Task #2 (1.50 pts.) - All Users Association Page

Progress: 100%

Details:

- Each line item should have a logical summary
- Each line item should include the username this entity is associated with
  - Clicking the username should redirect to that user's public profile
- Each line item should include a column that shows the total number of users the entity is associated with
- Each line item should have a link/button to a single view page of the entity
- Each line item should have a link/button to a delete action of the relationship (doesn't delete the entity or user, just the relationship)
- The page should have a section for stats (number of results and total number possible based on the query filters)
- The page should have logical options for filtering/sorting
  - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)
  - A filter with no matching records should show "no results available" or equivalent

Part 1:

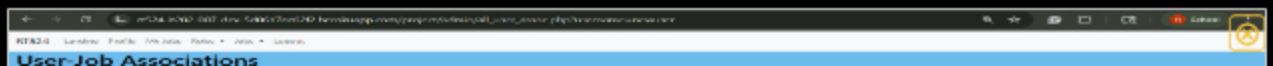
Progress: 100%

Details:

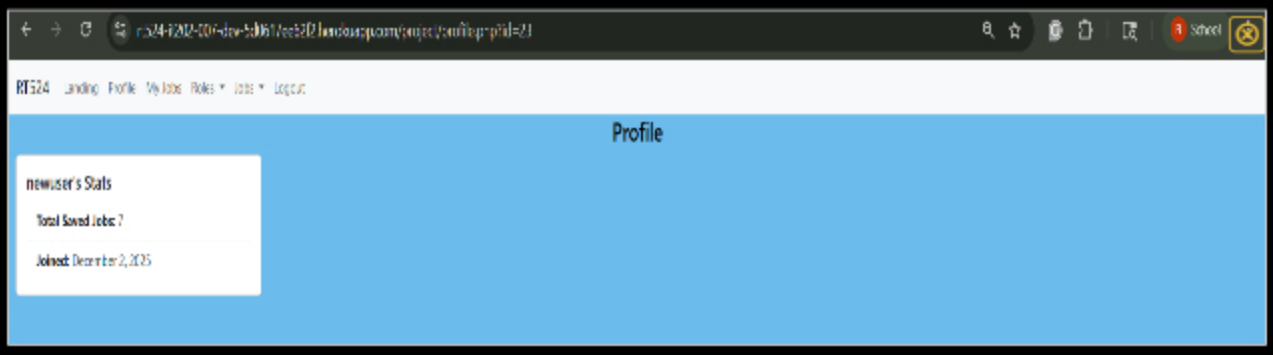
- Show a few examples of this page from heroku dev with various filters applied
- Ensure heroku dev url is visible
- Ensure each requirement is visible

all users association page

limit applied



username filter applied



public profile hyperlink

all users assoc page code

all users assoc code part 2

all users assoc page form

Saved: 12/6/2025 7:00:22 PM

Part 2:

Progress: 100%

Details:

- Describe how you solved showing the particular association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

The output was shown through join on my tables wherever the is_active piece was set to true. This was then displayed fully in the form with the username asosciated and total users with a count nested query in the select clause. The username filtering was done through the LIKE clause and the limit was also done through a form. The links on the page for deleting, profile, and viewing were linked to their respective pages with the delete page just setting the relationship is_active to 0 so it no longer appears.

https://rt524-it202-007-prod-a195f178bfec.herokuapp.com/project/admin/

Saved: 12/6/2025 7:00:22 PM

Task #3 (1.50 pts.) - Unassociated Page

Progress: 100%

Details:

- Each line item should have a logical summary
- Each line item should have a link/button to a single view page of the entity
- The page should have a section for stats (number of results and total number possible based on the query filters)
- The page should have logical options for filtering/sorting
  - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)
  - A filter with no matching records should show "no results available" or equivalent

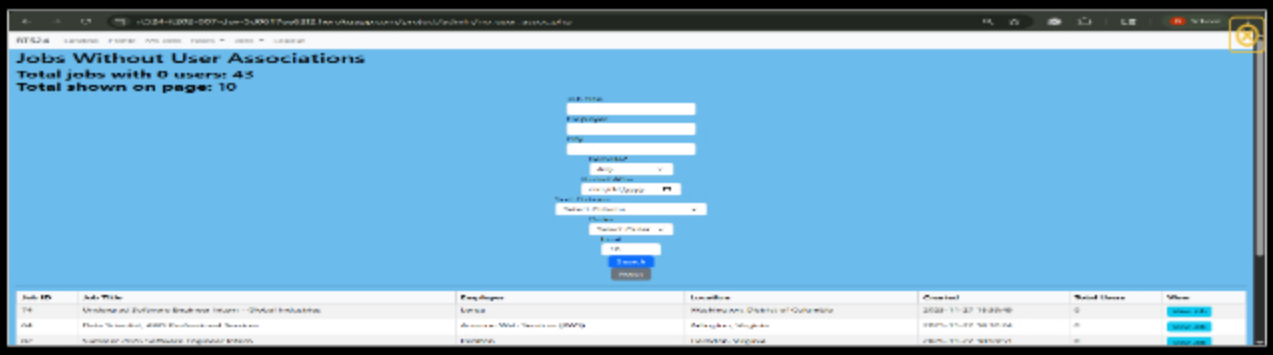
Part 1:

Progress: 100%

Details:

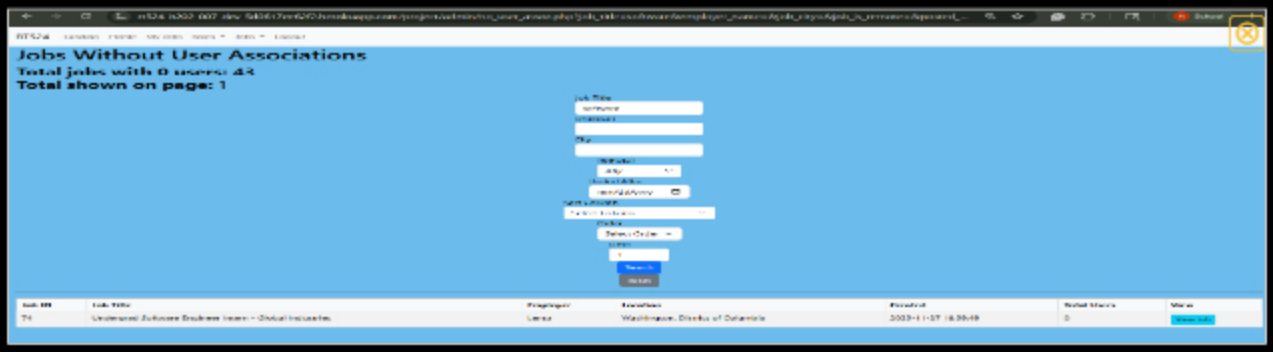
- Show a few examples of this page from heroku dev with various filters applied
- Ensure heroku dev url is visible
- Ensure each requirement is visible

- Ensure each requirement is visible



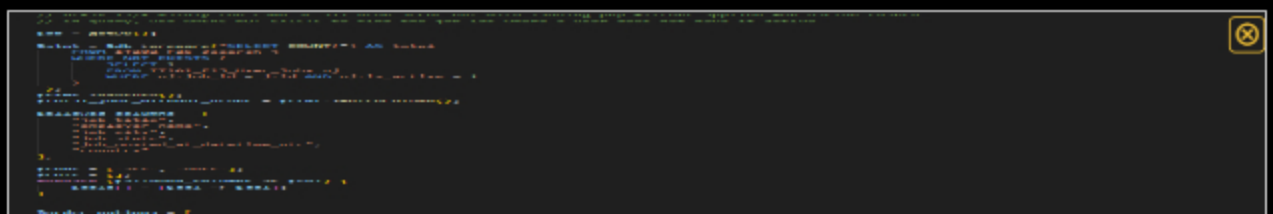
no users associated page

job title filter applied



limit applied

none found



```
[illegible]
```

unassociated code part 1

```
[illegible]
```

unassociated code part 2

```
[illegible]
```

unassociated part 3

```
[illegible]
```

unassociated form 1

```
[illegible]
```

unassociated form 2

Saved: 12/6/2025 7:08:14 PM

Part 2:

Progress: 100%

Details:

- Describe how you solved showing the particular association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

The query was essentially the same for this page but it was set to where the is_active was 0 to show the item was not associated instead with a where not exists clause to ensure no user had it associated with them at all rather than just a set amount of users. The filtering for each field was done through a form, the stats through an overall count and current count, and teh single view linked to the job details page.

https://rt524-it202-007-prod-a195f178bfec.herokuapp.com/project/admin/no_user_assoc.php

Saved: 12/6/2025 7:08:14 PM

Task #4 (1.50 pts.) - Admin Association Page (Like User Roles)

Progress: 100%

Details:

- The page should have a form with two fields
  - Partial match for username
  - Partial match for entity reference (name or something user-friendly)
- Submitting the form should give up to 25 matches of each
  - Likely best to show as two separate columns
- Each entity and user will have a checkbox next to them
- Submitting the checked associations should apply the association if it doesn't exist; otherwise it should remove the association
  - A filter with no matching records should show "no results available" or equivalent

Part 1:

Progress: 100%

Details:

- Show a few examples of this page from heroku dev with various selections having been submitted
- Ensure heroku dev url is visible
- Ensure each requirement is visible

assign jobs page

none found

both valid inputs

association toggled

apply associations button

assign page code part 1

assign query code

assign form 1

assign form part 2

Saved: 12/6/2025 7:08:46 PM

Part 2:

Progress: 100%

Details:

- Describe how your code solves the requirements; be clear

Your Response:

There are 2 forms that require an input, one for the job title and one for the username, both partial matched in the database. If both are provided, the query matches the user ids and the job id matching the title and does an INSERT query where the values are the ones provided in the form. If the item is already toggled on, the item will be toggled off.

https://rt524-it202-007-prod-a195f178bfec.herokuapp.com/project/admin/admin_association.php

Saved: 12/6/2025 7:08:46 PM

Section #3: (1 pt.) Misc

Progress: 100%

Task #1 (0.33 pts.) - Github Details

Progress: 100%

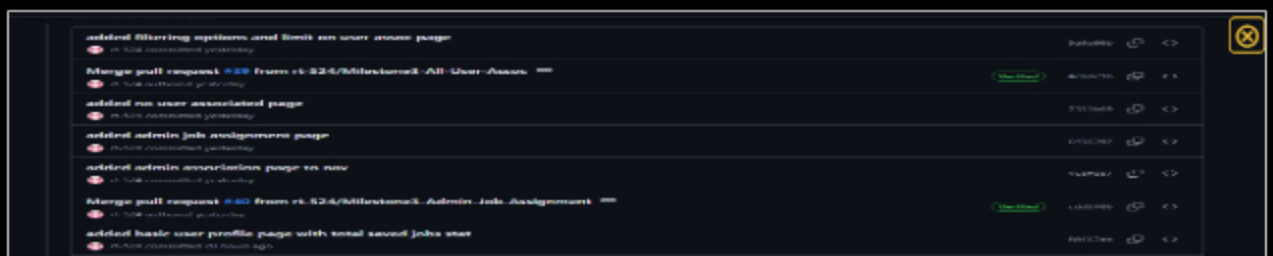
Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history

commits 1



commit 2

commits 3

Saved: 12/6/2025 7:09:15 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request for Milestone3 to dev (should end in `/pull/#`)

URL #1
https://github.com/rt-524/rt524-it202-007/pull/42

URL
https://github.com/rt-524/rt524-it

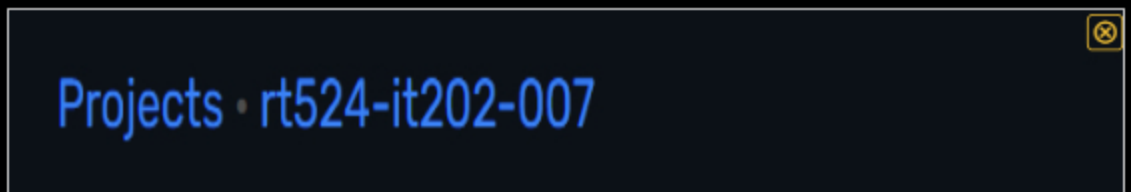
Saved: 12/6/2025 7:09:15 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



7 hrs 45 mins over the Last 7 Days in rt524-it202-007 under all branches.

wakatime overall

wakatime files

Saved: 12/6/2025 6:51:57 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

In this Milestone, I learned the general idea of association how to properly write queries for associations. For example, the biggest issue I ran into was dealing with duplicates in the no users associated page because I tried to go through all users where the job active was 0, but it was not working properly. I then decided to use the WHERE NOT EXISTS clause which helped me do a proper match and get all jobs that aren't associated to anyone.

Saved: 12/6/2025 6:53:25 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was making the filtering as this was mainly the same from the other pages. I also found the stats part to be easy as it was just counting the total rows that were returned first from the databse then after the filter was applied.

Saved: 12/6/2025 6:54:14 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was dealing with the forms and how to make them connect properly. For example, the checkbox form was not working for me for the longest time and I could not figure out the issue. I instead decided to remake the form by itself without using the functions I had before and that seemed to work. I manually created the apply button and had it submit the form with the checkbox status into the query.

Saved: 12/6/2025 6:56:31 PM